

Министерство образования Республики Беларусь

Учреждение образования
БЕЛОРУССКИЙ ГОСУДАРСТВЕННЫЙ УНИВЕРСИТЕТ
ИНФОРМАТИКИ И РАДИОЭЛЕКТРОНИКИ

Факультет компьютерного проектирования
Кафедра программного обеспечения информационных технологий

ПОЯСНИТЕЛЬНАЯ ЗАПИСКА

К курсовому проекту
на тему

ВЕБ-ПРИЛОЖЕНИЕ, РЕАЛИЗУЮЩЕЕ СЕТЕВУЮ ИГРУ «2048»

БГУИР КП 6-05-0612-01 009 ПЗ

Студент

Волков А.С.

Руководитель

Болтак С.В.

Минск 2026

СОДЕРЖАНИЕ

Введение	6
1 Выбор структуры системы, линии связи и структуры сигналов	8
1.1 Выбор структуры системы.....	8
1.2 Выбор линии связи	9
1.3 Структура сигналов (сообщений)	10
2 Алгоритм функционирования системы	12
2.1 Общий алгоритм работы	12
2.2 Алгоритм аутентификации	12
2.3 Алгоритм игрового процесса (клиент)	13
2.4 Алгоритм обработки запроса на сервере	14
3 Разработка структурной схемы системы.....	15
4 Расчёт частотных и временных параметров	16
4.1 Подготовка захвата	16
4.2 Частотные параметры.....	17
5 Выбор и энергетический расчёт линии связи	20
5.1 Выбор линии связи	20
5.2 Энергетический расчёт (качественная оценка).....	20
6 Выбор элементной базы системы.....	21
6.1 Аппаратная элементная база.....	21
6.2 Программная элементная база.....	21
7 Проектирование принципиальной электрической схемы системы	23
8 Системные расчёты	24
8.1 Скорость передачи сообщений.....	24
8.2 Пропускная способность канала связи	24
8.3 Спектр сигнала в линии связи	25
8.4 Помехоустойчивость	25

8.5 Надёжность.....	26
9 Разработка программного обеспечения.....	27
9.1 Серверная часть	27
9.2 Клиентская часть	27
9.3 Тестирование.....	28
Заключение.....	31
Список использованных источников.....	32
Приложение А. Листинг кода.....	33
Приложение Б. Схема алгоритма работы системы	42

ВВЕДЕНИЕ

Сетевые информационные системы составляют основу современного программного обеспечения: от мобильных приложений до корпоративных порталов взаимодействие компонентов осуществляется по каналам связи с использованием стандартизированных протоколов [1]. Дисциплина «Компьютерные системы и сети» формирует навыки анализа структуры таких систем, выбора линий связи, оценки временных и частотных характеристик, а также разработки прикладного ПО, использующего сокетный и прикладной уровни стека ТСР/ІР.

Игра «2048» — популярная логическая головоломка на поле 4×4; сетевая реализация предполагает хранение профилей игроков, таблицы рекордов и обмен данными между браузером (клиент) и сервером по протоколу HTTP [2]. Аналоги — веб-версии 2048 с локальным хранением лучшего результата; в разрабатываемом проекте добавлены регистрация, JWT-аутентификация [3], серверная БД и REST-хранилище файлов (аватары, сохранения партий) [4], что расширяет учебную модель до полноценной клиент-серверной системы.

Цель курсового проектирования — разработка веб-приложения, реализующего сетевую игру «2048», с обоснованием структуры системы, алгоритмов, анализом сетевого трафика в Wireshark и программной реализацией на заданном стеке технологий.

Задачи работы:

- выбрать структуру системы, линию связи и форматы сообщений;
- описать алгоритмы функционирования клиента и сервера;

- описать состав системы и сетевое взаимодействие компонентов (диаграмма сетей);
- выполнить анализ временных и частотных параметров обмена с помощью Wireshark;
- обосновать выбор элементной базы (аппаратных и программных средств);
- выполнить системный анализ (скорость передачи, пропускная способность, надёжность) по данным Wireshark;
- реализовать и протестировать программное обеспечение.

Принципы проектирования: клиент-серверная архитектура; stateless-аутентификация на JWT; REST API; разделение игровой логики (клиент) и персистентных данных (сервер); контейнеризация развёртывания (Docker, nginx).

1 ВЫБОР СТРУКТУРЫ СИСТЕМЫ, ЛИНИИ СВЯЗИ И СТРУКТУРЫ СИГНАЛОВ

1.1 Выбор структуры системы

Для веб-приложения с централизованным хранением данных выбрана клиент-серверная структура с одним логическим сервером приложений и выделенным сервером базы данных.

Уровни системы:

1. Клиентский уровень — веб-браузер; выполняет отображение UI, игровую логику 2048, формирование HTTP-запросов к API.
2. Серверный уровень — приложение Spring Boot [5]: REST API, безопасность, бизнес-логика, файловое хранилище.
3. Уровень данных — СУБД PostgreSQL [6] (таблицы пользователей и рекордов); файловая система (аватары, JSON-сохранения партий).
4. Промежуточный уровень (опционально) — reverse proxy nginx [7] (проксирование HTTP, заголовки `X-Real-IP`, `X-Forwarded-For`).

Альтернатива — одноранговая (P2P) структура: узлы обмениваются сообщениями напрямую по TCP/UDP. Для игры с общей таблицей рекордов и единой БД P2P не обеспечивает согласованности данных; клиент-серверная модель проще в сопровождении и соответствует заданию.

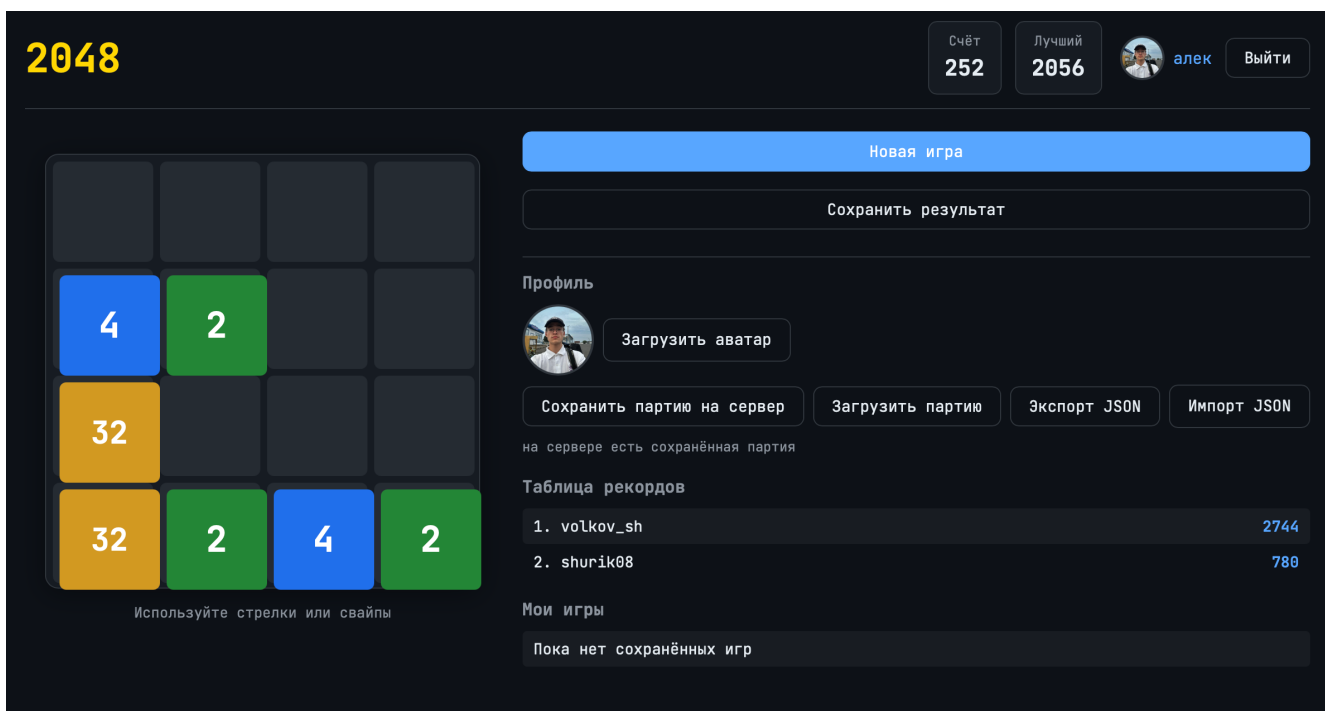


Рисунок 1 — общий вид главной страницы игры в браузере

1.2 Выбор линии связи

В качестве физической линии связи рассматривается канал локальной сети Ethernet (витая пара категории не ниже 5е) или беспроводной канал Wi-Fi (IEEE 802.11). На канальном уровне — кадры Ethernet; на сетевом — IP; на транспортном — TCP; на прикладном — HTTP/1.1.

Логическая линия связи между клиентом и сервером — TCP-соединение, инициируемое браузером к порту 80 (nginx) или 8080 (Spring Boot) узла сервера. При развёртывании в Docker [8] трафик внутри хоста проходит через виртуальную сеть bridge.

Параметры типового канала (для анализа в разделе 4):

- пропускная способность канала — до 100 Мбит/с (Fast Ethernet / Wi-Fi);

- ожидаемая задержка в локальной сети RTT — порядка 1–5 мс (уточняется по захвату Wireshark [9]).

1.3 Структура сигналов (сообщений)

Под «структурой сигналов» в цифровой системе понимают формат и состав передаваемых сообщений.

1. HTTP-запрос/ответ — текстовые заголовки + тело (при наличии). Пример запроса регистрации:

```
http
POST /api/auth/register HTTP/1.1
Host: localhost
Content-Type: application/json

{"username":"player1","password":"secret123"}
```

2. JSON — структура данных API (UTF-8). Поля регистрации: «username», «password»; ответ: «token», «username», «userId».

3. JWT — компактный подписанный токен (заголовок, полезная нагрузка, подпись, Base64URL). В полезной нагрузке: «sub» (имя пользователя), «userId», срок действия «exp».

4. Multipart/form-data — загрузка аватара («file»).

5. JSON-состояние игры (файловое хранилище) — поля: «grid», «score»,

«bestScore», «movesCount», «won», «over», «maxTile».

№	Направление	Метод и путь	Формат тела	Назначение
1	Клиент → Сервер	POST /api/auth/register	JSON	Регистрация
2	Клиент → Сервер	POST /api/auth/login	JSON	Вход
3	Клиент → Сервер	POST /api/records	JSON + Bearer JWT	Сохранение рекорда
4	Клиент → Сервер	GET /api/records/top	—	Таблица лидеров
5	Клиент → Сервер	PUT /api/files/avatars/me	multipart	Аватар
6	Клиент → Сервер	PUT /api/files/saves/me	JSON	Сохранение партии
7	Сервер → Клиент	HTTP 200/201/4xx	JSON или файл	Ответ

Таблица 1 — Основные типы сообщений системы

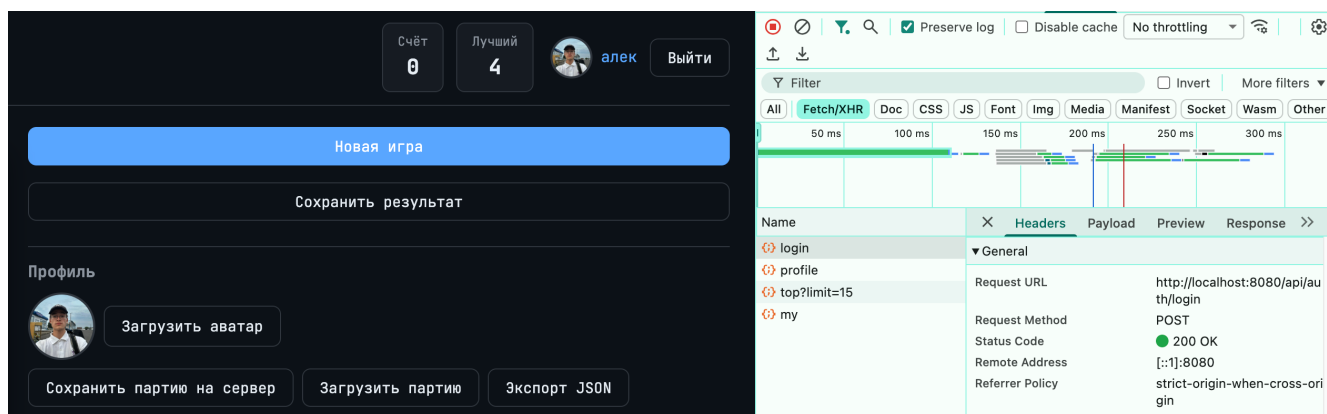


Рисунок 2 — вкладка Network в DevTools (пример POST /api/auth/login)

2 АЛГОРИТМ ФУНКЦИОНИРОВАНИЯ СИСТЕМЫ

2.1 Общий алгоритм работы

1. Пользователь открывает URL приложения в браузере.
2. Браузер загружает HTML, CSS, JavaScript (статические ресурсы).
3. Клиент инициализирует игровое поле, подписывается на события клавиатуры/касания.
4. При необходимости сетевого взаимодействия клиент формирует HTTP-запрос; при авторизованных операциях добавляет заголовок «Authorization: Bearer <token>».
5. Запрос проходит через nginx (если используется) на Spring Boot.
6. Цепочка фильтров сервера: контроль доступа (IP, rate limit) → журналирование → JWT-аутентификация → контроллер → сервис → БД/файлы.
7. Сервер формирует HTTP-ответ; клиент обновляет интерфейс.

2.2 Алгоритм аутентификации

1. Клиент отправляет «POST /api/auth/login» с учётными данными.
2. Сервер находит пользователя в БД, проверяет пароль (BCrypt).
3. При успехе генерируется JWT, возвращается клиенту.
4. Клиент сохраняет токен в «localStorage», использует в последующих запросах.
5. «JwtAuthFilter» на сервере извлекает токен, проверяет подпись и срок, устанавливает контекст безопасности.

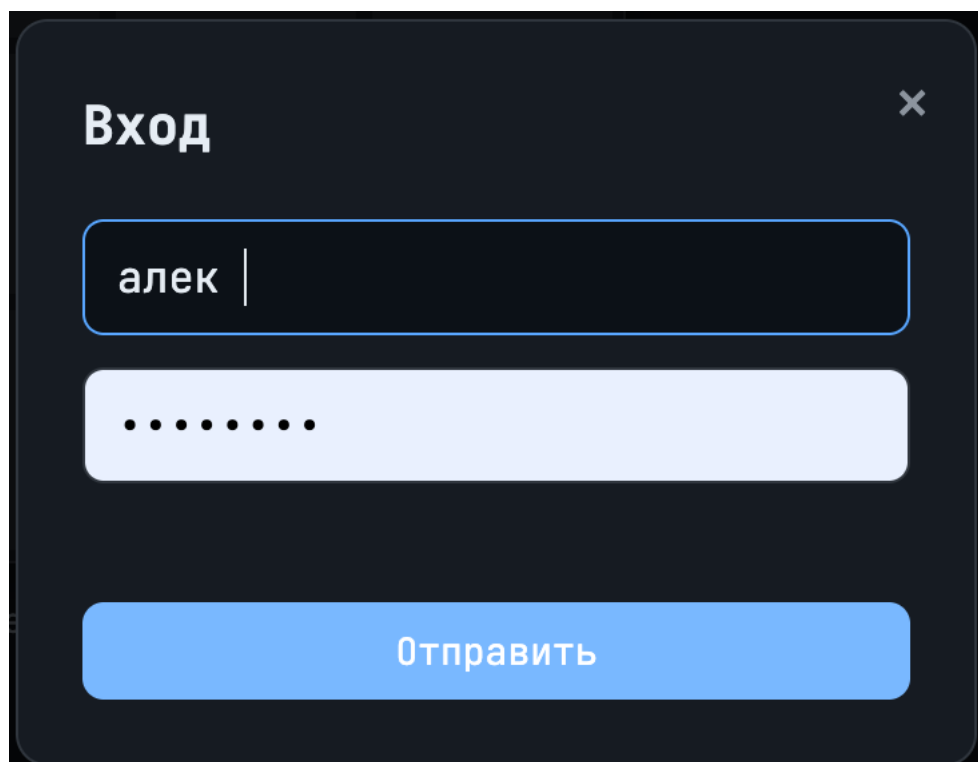


Рисунок 3 — модальное окно входа

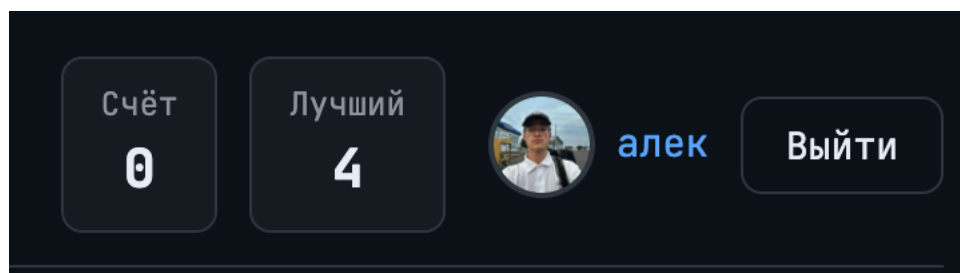


Рисунок 4 — успешный вход с отображением имени пользователя

2.3 Алгоритм игрового процесса (клиент)

1. «start()» — инициализация сетки $4 * 4$, две случайные плитки (2 или 4).
2. При нажатии стрелки или свайпе — «move(direction)»: сдвиг и слияние плиток, начисление очков.

3. При отсутствии ходов — «game over».
4. При достижении плитки 2048 — «overlay» победы (возможность продолжить).
5. По кнопке «Сохранить результат» — «POST /api/records» с текущим счётом (только для авторизованных).

Игровая логика выполняется на клиенте для минимизации сетевого трафика; сервер хранит итоги и опционально полное состояние партии.

2.4 Алгоритм обработки запроса на сервере

1. «AccessControlFilter»: проверка IP по чёрному списку; для «/api/auth/*» — учёт «rate limit».
2. «ApiLoggingFilter»: фиксация метода, URI, статуса, времени, пользователя.
3. «JwtAuthFilter»: разбор «Bearer»-токена.
4. «DispatcherServlet» → контроллер → сервис → репозиторий «JPA» «FileStorageService».
5. Формирование ответа; при ошибках — «GlobalExceptionHandler» .

```

2026-05-20T16:27:24.412+03:00 INFO 3003 --- [game-2048] [nio-8080-exec-4] http.access : GET /api/records/my 403 20ms user=-
2026-05-20T16:27:24.412+03:00 INFO 3003 --- [game-2048] [nio-8080-exec-1] http.access : GET /api/profile 403 20ms user=-
2026-05-20T16:27:24.473+03:00 INFO 3003 --- [game-2048] [nio-8080-exec-5] http.access : GET /api/records/top 200 81ms user=-
2026-05-20T16:27:49.458+03:00 INFO 3003 --- [game-2048] [io-8080-exec-10] http.access : GET /api/records/top 200 11ms user=-
2026-05-20T16:30:32.396+03:00 INFO 3003 --- [game-2048] [nio-8080-exec-3] http.access : POST /api/auth/login 400 35ms user=-
2026-05-20T16:30:41.591+03:00 INFO 3003 --- [game-2048] [nio-8080-exec-1] http.access : GET /api/records/top 200 6ms user=-
2026-05-20T16:31:02.223+03:00 INFO 3003 --- [game-2048] [nio-8080-exec-5] http.access : POST /api/auth/login 400 6ms user=-
2026-05-20T16:33:30.778+03:00 INFO 3003 --- [game-2048] [nio-8080-exec-9] http.access : POST /api/auth/login 200 107ms user=-
2026-05-20T16:33:30.875+03:00 INFO 3003 --- [game-2048] [nio-8080-exec-1] http.access : GET /api/records/top 200 16ms user=-
2026-05-20T16:33:30.910+03:00 INFO 3003 --- [game-2048] [nio-8080-exec-5] http.access : GET /api/records/my 200 51ms user=-
2026-05-20T16:33:30.910+03:00 INFO 3003 --- [game-2048] [nio-8080-exec-3] http.access : GET /api/profile 200 52ms user=-
2026-05-20T16:33:31.064+03:00 INFO 3003 --- [game-2048] [nio-8080-exec-9] http.access : GET /api/files/avatars/3 200 19ms user=-
2026-05-20T16:38:32.333+03:00 INFO 3003 --- [game-2048] [nio-8080-exec-4] http.access : GET /api/records/top 200 11ms user=-
2026-05-20T16:39:07.344+03:00 INFO 3003 --- [game-2048] [nio-8080-exec-3] http.access : POST /api/auth/login 200 106ms user=-
2026-05-20T16:39:07.456+03:00 INFO 3003 --- [game-2048] [nio-8080-exec-5] http.access : GET /api/records/top 200 21ms user=-
2026-05-20T16:39:07.469+03:00 INFO 3003 --- [game-2048] [io-8080-exec-10] http.access : GET /api/profile 200 33ms user=-
2026-05-20T16:39:07.469+03:00 INFO 3003 --- [game-2048] [nio-8080-exec-4] http.access : GET /api/records/my 200 36ms user=-
2026-05-20T16:39:07.523+03:00 INFO 3003 --- [game-2048] [nio-8080-exec-6] http.access : GET /api/files/avatars/3 200 25ms user=-

```

Рисунок 5 — фрагмент лога сервера с записями http.access после нескольких API-запросов

3 РАЗРАБОТКА СТРУКТУРНОЙ СХЕМЫ СИСТЕМЫ

Структурная схема (ГОСТ 2.701 [10], СТП 01-2024 [11], п. 3.7) отражает состав частей системы и связи между ними. Для веб-приложения «2048» используется клиент-серверная архитектура, обоснованная в п. 1.1.

Компонент	Связь с	Протокол / интерфейс
Браузер	nginx или Spring Boot	HTTP, TCP
nginx	Spring Boot :8080	HTTP
Spring Boot	PostgreSQL, storage_data	JDBC, файловая система
PostgreSQL	Spring Boot	TCP, порт 5432
storage_data	Spring Boot	REST «/api/files/...»

Таблица 2 — Состав и связи компонентов системы

Графическое представление топологии (узлы, порты, протоколы) выполнено в виде диаграммы сетевого взаимодействия (приложение А).

4 РАСЧЁТ ЧАСТОТНЫХ И ВРЕМЕННЫХ ПАРАМЕТРОВ

Оценка параметров обмена выполнена «экспериментально» с помощью анализатора сетевого трафика «Wireshark», без аналитических формул. Захват ведётся на интерфейсе, через который клиент обращается к серверу (обычно «lo0» на macOS или «Loopback» при «localhost», либо активный Ethernet/Wi-Fi при доступе по IP в ЛВС).

4.1 Подготовка захвата

1. Запустить приложение («docker compose up» или «mvn spring-boot:run»).
2. Открыть Wireshark, выбрать интерфейс (для «http://localhost:8080» — «loopback»).
3. Задать фильтр отображения (после захвата): «tcp.port == 8080 || tcp.port == 80» (при nginx — порт 80).
4. Начать захват (Start), в браузере выполнить сценарий: вход → загрузка таблицы рекордов → сохранение рекорда → (опционально) загрузка аватара.
5. Остановить захват, сохранить файл «.pcapng» (при необходимости приложить к отчёту).

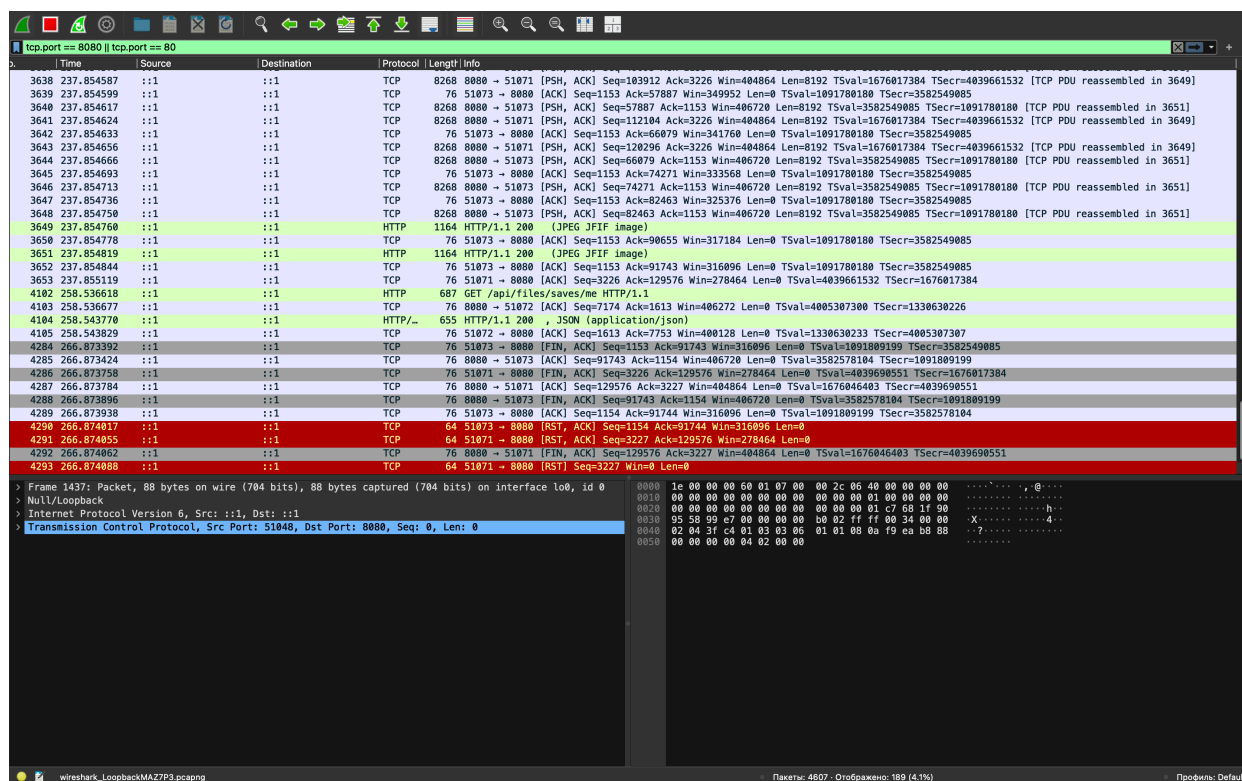


Рисунок 5 — Wireshark список пакетов с фильтром tcp.port == 8080 || tcp.port == 80

4.2 Частотные параметры

Для HTTP/1.1 поверх TCP полное время взаимодействия «запрос — ответ» удобно разбирать по полям статистики Wireshark и по цепочке пакетов одного TCP-потока.

Методика:

- выделить HTTP-запрос (например, «POST /api/auth/login»);
- найти соответствующий ответ («HTTP/1.1 200»);
- в «Statistics → Flow Graph» или по столбцу «Time» оценить интервал между запросом и началом ответа;
- для TCP при первом обращении учесть трёхстороннее рукопожатие (SYN, SYN-ACK, ACK) — отдельные строки в трассировке;

- в «Analyze → Expert Information» просмотреть предупреждения (повторные передачи, задержки).

Операция	Размер кадра / полезной нагрузки (по Wireshark)	Интервал запрос → ответ, мс	Примечание
GET/api/records/top	ответ ~1–3 КБ	30–80	чтение из БД
POST/api/auth/login	запрос ~0,3–0,6 КБ	80–250	заметна задержка BCrypt на сервере
POST /api/records	запрос ~0,2–0,4 КБ	25–70	короткий JSON
PUT /api/files/avatars/me	тело до 1 МБ	100–600	крупный multipart

Таблица 3 — Временные характеристики по данным Wireshark (пример для локального стенда; подставить свои значения из захвата)

Выводы по времени: в локальной сети (или на loopback) доминирует время обработки на сервере (аутентификация, запросы к PostgreSQL), а не задержка распространения в канале. Для «POST /api/auth/login» интервал «запрос — ответ» обычно максимален среди перечисленных операций.

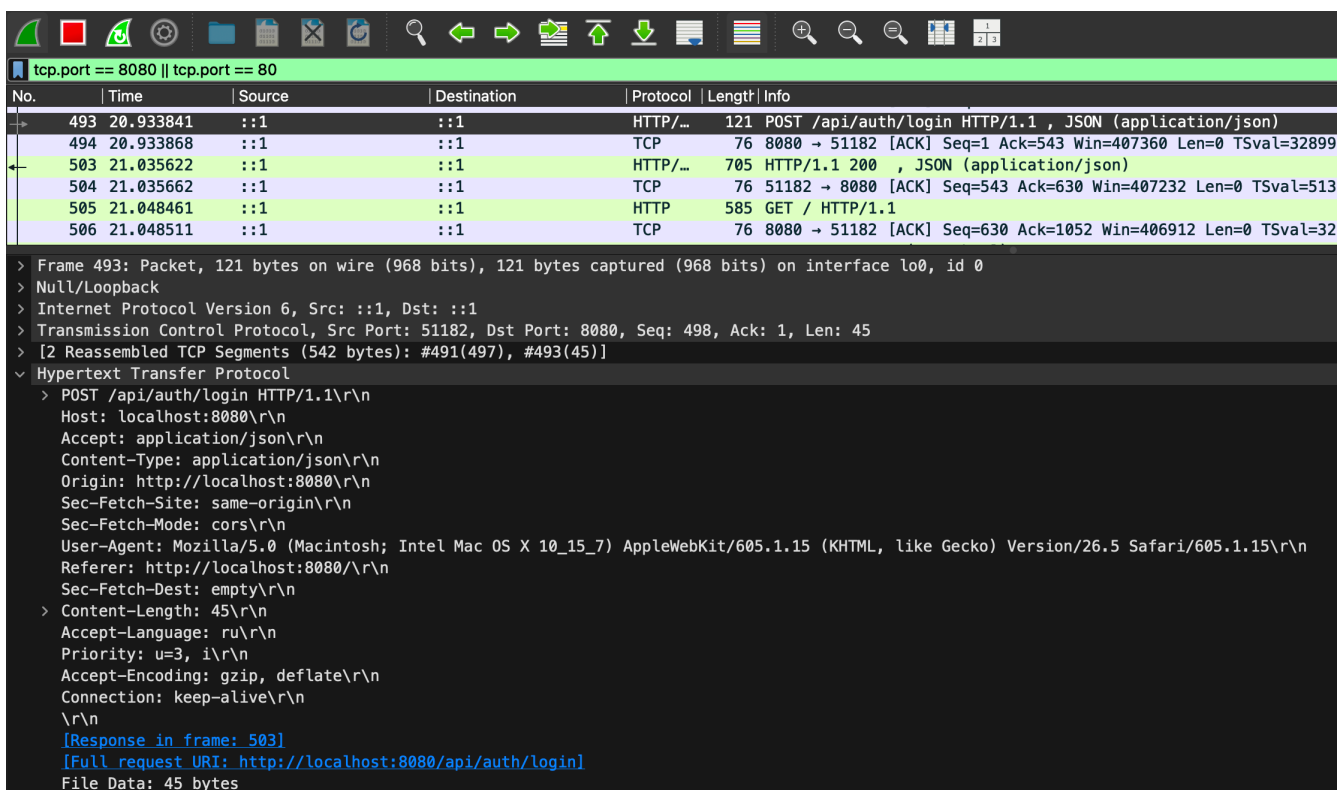


Рисунок 6 — Wireshark развёрнутый HTTP-запрос «POST /api/auth/login» и соседний пакет ответа 200 OK

5 ВЫБОР И ЭНЕРГЕТИЧЕСКИЙ РАСЧЁТ ЛИНИИ СВЯЗИ

5.1 Выбор линии связи

Принята линия связи «клиент — сервер» по локальной сети TCP/IP с физической средой Ethernet или Wi-Fi. Обоснование: веб-приложение не требует выделенных цифровых каналов; стандартная ИТ-инфраструктура университета/дома обеспечивает достаточную полосу и надёжность.

5.2 Энергетический расчёт (качественная оценка)

Энергетические характеристики линии связи для веб-приложения в ЛВС оцениваются качественно, без расчётных формул.

Основное потребление приходится на ПК клиента и сервер приложений (CPU, ОЗУ, диск), а не на передачу отдельных HTTP-сообщений. По данным Wireshark (раздел 4) обмен короткими всплесками; во время игрового процесса сетевой активности почти нет. Следовательно, доля энергии, затрачиваемой именно на передачу битов по Ethernet/Wi-Fi, мала по сравнению с энергией вычислений (обработка JWT, BCrypt, запросы к PostgreSQL).

Вывод: выбранная линия связи (Ethernet/Wi-Fi, TCP/IP) по энергетическим показателям подходит для учебного стенда; узким местом является обработка на сервере, а не мощность канала.

6 ВЫБОР ЭЛЕМЕНТНОЙ БАЗЫ СИСТЕМЫ

Таблицы раздела оформлены по ГОСТ 2.105-95 [12].

6.1 Аппаратная элементная база

Компонент	Требования	Пример
Клиентский ПК / смартфон	ОЗУ ≥ 4 ГБ, браузер с ES6	ПК с Windows 10/11, macOS, Android
Сервер разработки	ОЗУ ≥ 8 ГБ, 2 ядра CPU	ПК разработчика / ВМ
Сетевое оборудование	100 Мбит/с Ethernet или Wi-Fi 802.11n/ac	Коммутатор / роутер ЛВС

6.2 Программная элементная база

Назначение	Продукт	Версия
ОС сервера	Linux / macOS / Windows	—
Среда выполнения	Eclipse Temurin JDK	17
Framework	Spring Boot	3.2.5
СУБД	PostgreSQL	15
Reverse proxy	nginx	1.27
Контейнеризация	Docker, Docker Compose	актуальная
Анализ трафика	Wireshark	актуальная
Клиент	HTML5, CSS3, JavaScript (ES6)	—
Сборка	Apache Maven	3.9+

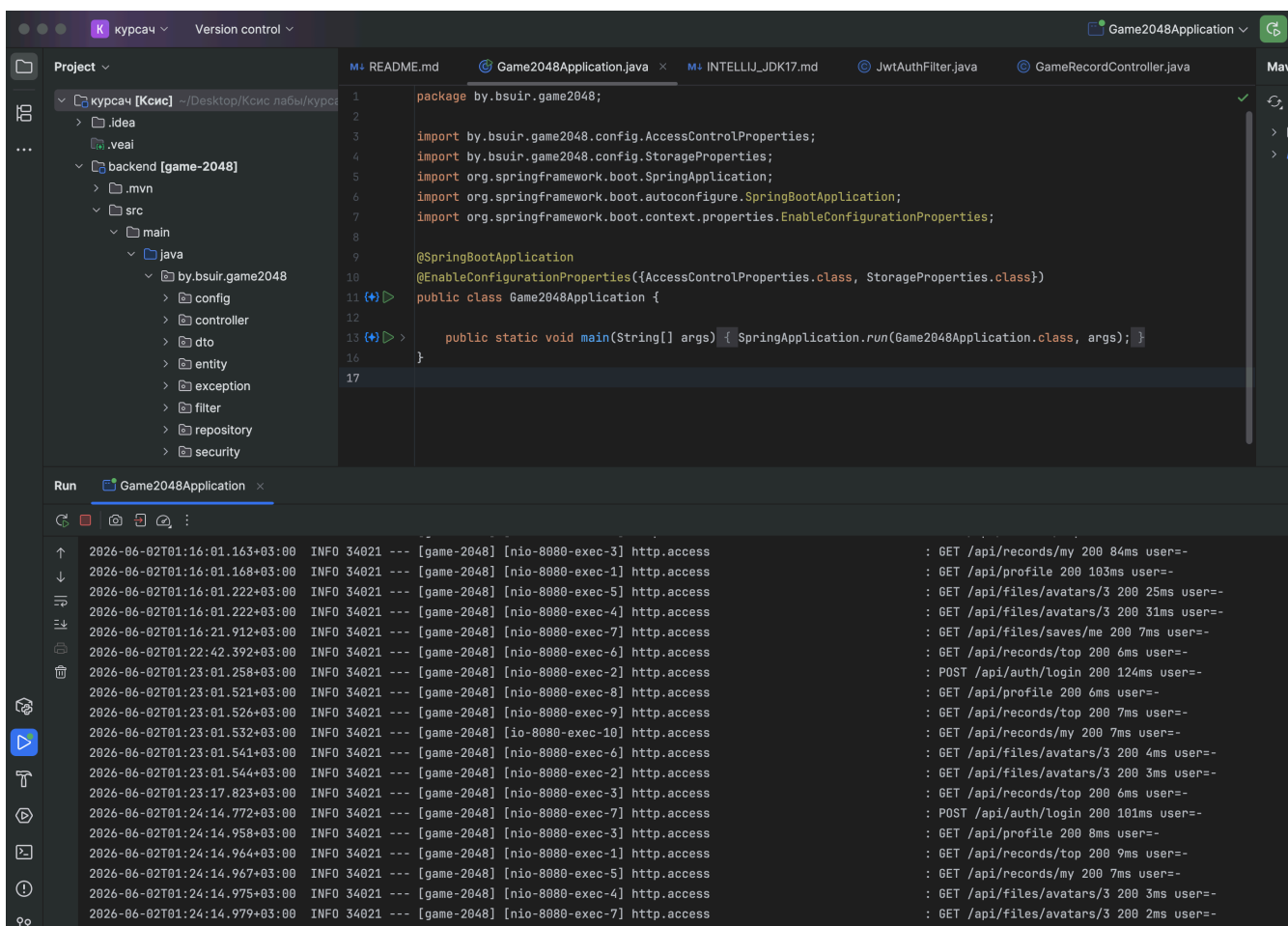


Рисунок 7 — окно IntelliJ IDEA с запущенным Game2048Application

7 ПРОЕКТИРОВАНИЕ ПРИНЦИПИАЛЬНОЙ ЭЛЕКТРИЧЕСКОЙ СХЕМЫ СИСТЕМЫ

В соответствии с заданием (п. 7 — принципиальная электрическая схема) для программно-сетевой системы приводится описание соединений (развёртывание) компонентов и используемых сетевых портов. Детальная диаграмма узлов и линий связи совмещена с графическим материалом по заданию (п. 5) и приведена на рисунке А.1 (приложение А).

Узел	Сервис	Порт	Назначение	Соединение
Хост	Браузер пользователя	—	Клиент	→ «http://localhost/»
Контейнер	nginx	80	HTTP-вход	→ app:8080
Контейнер	app (Spring Boot)	8080	REST API, статика	→ db:5432, том «storage_data»
Контейнер	db (PostgreSQL)	5432	БД	← JDBC от app
Том Docker	pgdata	—	Данные БД	монтируется в db
Том Docker	storage_data	—	Файлы avatars/saves	монтируется в app

Таблица 4 — Соединения при развёртывании (Docker Compose)

Внешний пользователь обращается к «nginx» (порт 80); приложение доступно напрямую на 8080 для отладки. Внутри сети Docker контейнеры «app» и «db» обмениваются данными по именам сервисов из «docker-compose.yml».

8 СИСТЕМНЫЕ РАСЧЁТЫ: СКОРОСТИ ПЕРЕДАЧИ СООБЩЕНИЙ, ПРОПУСКНОЙ СПОСОБНОСТИ КАНАЛА СВЯЗИ СПЕКТРА СИГНАЛА В ЛИНИИ СВЯЗИ, РАСЧЁТ ПОМЕХОУСТОЙЧИВОСТИ, РАСЧЁТ НАДЁЖНОСТИ

Раздел выполнен на основе того же захвата трафика, что и в разделе 4 (файл «.pcapng»). Численные оценки получены из полей Wireshark, без аналитических формул.

8.1 Скорость передачи сообщений

Методика: для выбранного HTTP-диалога (например, «POST /api/records») определить размер полезной нагрузки приложения (поле «Length» у пакетов с HTTP, сумма сегментов TCP с данными запроса и ответа) и время между первым байтом запроса и последним байтом ответа (по меткам «Time»). Удобно использовать «Statistics → Conversations → TCP» — столбцы Bytes и Duration.

Пример интерпретации (подставить значения из своего захвата):

- сохранение рекорда: полезный объём порядка сотен байт, длительность обмена десятки миллисекунд — средняя скорость на уровне приложения;
- загрузка аватара: объём до 1 МБ, длительность сотни миллисекунд — кратковременный всплеск трафика, в Wireshark видна серия крупных TCP-сегментов.

8.2 Пропускная способность канала связи

Сравнение с номиналом канала (100 Мбит/с Ethernet / Wi-Fi): пики трафика одного клиента занимают малую долю полосы. Узкое место — серверная обработка и диск, а не пропускная способность линии.

8.3 Спектр сигнала в линии связи

Wireshark отображает цифровые кадры Ethernet и пакеты IP/TCP/HTTP; спектр радиосигнала Wi-Fi или спектр линейного кода Ethernet не восстанавливается из PCAP-файла прикладного захвата.

Вывод для пояснительной записки: спектральные характеристики физической линии задаются стандартом PHY (Ethernet/Wi-Fi); прикладные HTTP-сообщения передаются как двоичные данные поверх готового канала. Для учебного проекта достаточно констатации соответствия стандартам ЛВС и отсутствия узкого места на физическом уровне.

8.4 Помехоустойчивость

Анализ в Wireshark:

- «Analyze → Expert Information» — наличие или отсутствие «Retransmission», «Duplicate ACK», «Out-of-Order»;
- при стабильном loopback/ЛВС повторных передач нет или их единичное число;
- на прикладном уровне — коды «HTTP 200/201» при успешных операциях, «401/403» при ошибках авторизации (ожидаемое поведение).

Механизмы повышения устойчивости в системе: контрольная сумма TCP, валидация JSON, JWT-подпись, BCrypt для паролей, «StoragePathResolver» (защита путей), «rate limit» в «security-access.json».

8.5 Надёжность

По трассировке за сеанс (30–60 мин работы с приложением):

- подсчитать HTTP-запросы с ответами 5xx (ошибки сервера) — для исправно работающего стенда их не должно быть;
- зафиксировать обрывы TCP RST, FIN без ответа) — на loopback отсутствуют;
- сопоставить с логами «http.access» на сервере (раздел 2.4).

Цепочка компонентов: клиент → сеть → nginx → Spring Boot → PostgreSQL. Отказ любого звена приводит к ошибкам в Wireshark (таймаут, RST, HTTP 502/503). Для локального развёртывания наблюдается устойчивая работа при штатных сценариях.

Показатель	Результат анализа
Интенсивность HTTP-трафика	эпизодическая, всплески при входе/рекордах
Загрузка канала ЛВС	низкая, запас по полосе большой
Повторные передачи TCP	не наблюдаются (или единичны) на loopback
Ошибки HTTP 5xx	отсутствуют при штатной работе
Узкое место	обработка на сервере (login, БД), не канал

Таблица 5 — Сводка системного анализа (Wireshark)

9 РАЗРАБОТКА ПРОГРАММНОГО ОБЕСПЕЧЕНИЯ

9.1 Серверная часть

Модули:

- «AuthController», «AuthService» — регистрация/вход, BCrypt, выдача JWT;
- «GameRecordController», «GameRecordService» — сохранение и выборка рекордов;
- «FileStorageController», «FileStorageService» — REST-хранилище;
- «ProfileController» — сведения о профиле;
- «ApiLoggingFilter», «AccessControlFilter» — журналирование и фильтрация;
- «SecurityConfig», «JwtAuthFilter» — правила доступа к «/api/**».

Сущности БД: «User» (id, username, passwordHash, createdAt); «GameRecord» (user, score, maxTile, playedAt, movesCount).

9.2 Клиентская часть

- «game.js» — логика 2048, «getState()», «loadState()»;
- «api.js» — обёртка «fetch» для REST;
- «app.js» — UI, таблица рекордов, профиль, аватар, сохранение партии;
- «index.html», «styles.css» — адаптивная вёрстка (CSS Grid, «clamp»).

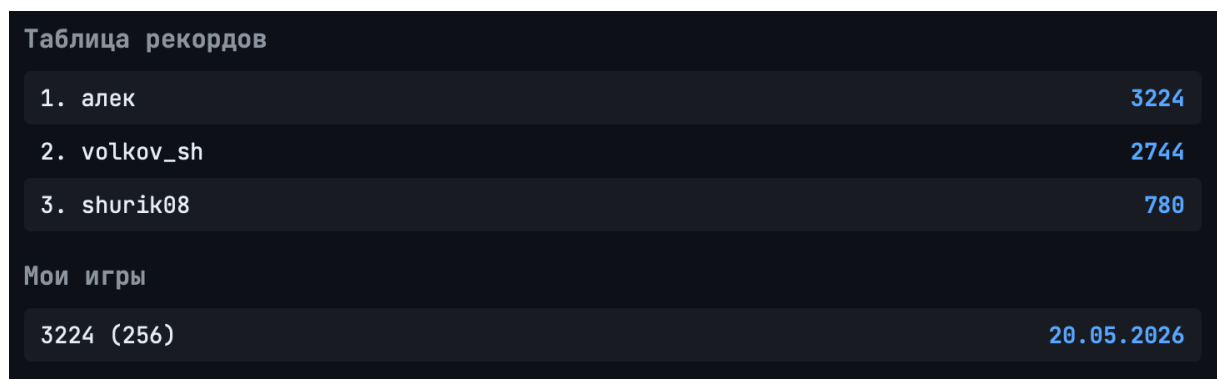


Рисунок 8 — таблица рекордов и блок «Мои игры» после сохранения результата

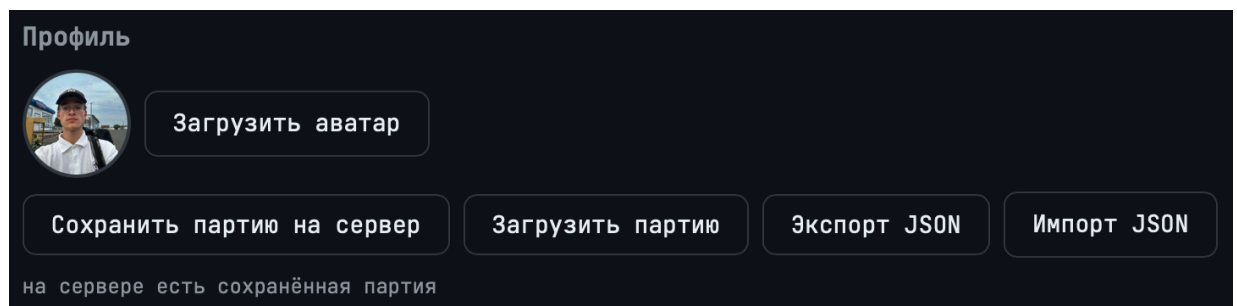


Рисунок 9 — блок «Профиль» — аватар, кнопки сохранения/загрузки партии

9.3 Тестирование

№	Сценарий	Ожидаемый результат	Статус
1	Регистрация нового пользователя	HTTP 200, JWT в ответе	✓
2	Вход с верным паролем	HTTP 200, токен	✓
3	Сохранение рекорда без токена	HTTP 401	✓
4	GET /api/records/top	JSON-список	✓
5	Загрузка аватара	HTTP 200, файл в	✓

		storage	
6	PUT/GET сохранения партии	Восстановление поля	✓
7	Превышение rate limit auth	HTTP 403	✓
8	Отображение на ширине 360 px	Корректная вёрстка	✓

Таблица 6 — Результаты функционального тестирования

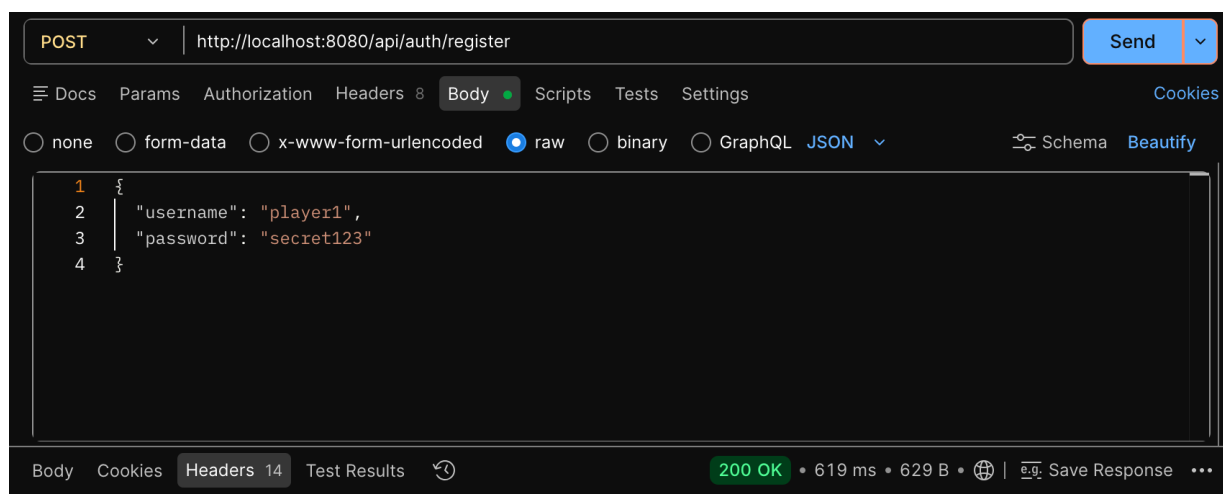


Рисунок 10 — Postman register

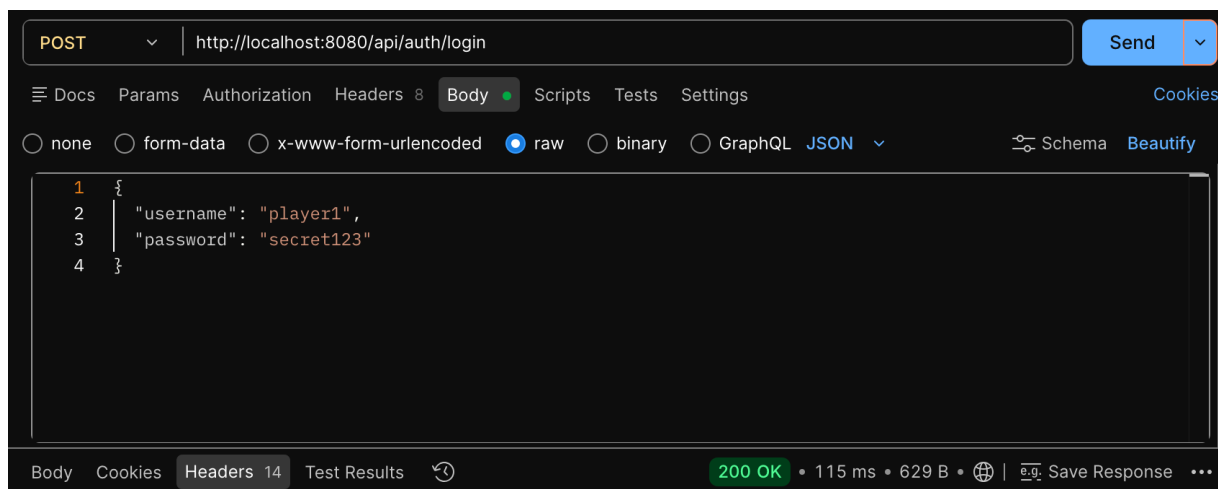


Рисунок 11 — Postman login

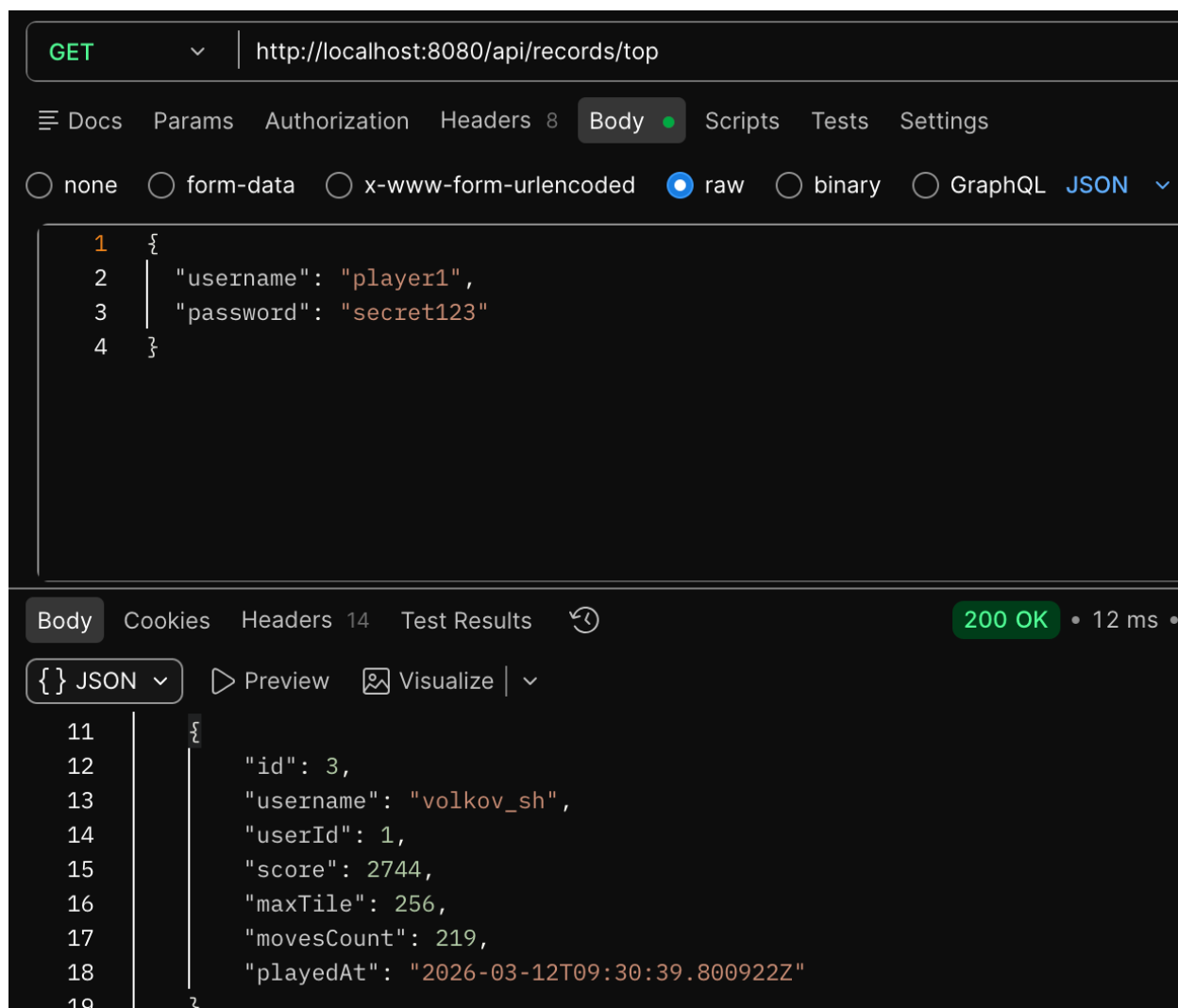


Рисунок 12 — Postman records

ЗАКЛЮЧЕНИЕ

В ходе курсового проектирования разработано веб-приложение, реализующее сетевую игру «2048», отвечающее заданию по дисциплине «Компьютерные системы и сети».

Основные результаты:

- обоснована и реализована клиент-серверная структура с линией связи TCP/IP и прикладным протоколом HTTP;
- определена структура сигналов: JSON, JWT, multipart, статические ресурсы;
- описаны алгоритмы аутентификации, игрового процесса и обработки запросов;
- разработаны структурная схема и схема соединений (приложения В, Д);
- выполнен анализ временных параметров, пропускной способности, помехоустойчивости и надёжности по данным Wireshark;
- выбрана элементная база (JDK 17, Spring Boot, PostgreSQL, nginx, Docker);
- реализовано программное обеспечение с JWT, таблицей рекордов, файловым хранилищем, журналированием и фильтрацией запросов;
- проведено функциональное тестирование основных сценариев.

Поставленная цель достигнута. Разработанная система может использоваться как учебный стенд для демонстрации сетевого взаимодействия веб-клиента и REST API и как основа для дальнейшего расширения.

СПИСОК ИСПОЛЬЗОВАННЫХ ИСТОЧНИКОВ

1. Таненбаум, Э. Компьютерные сети / Э. Таненбаум, Д. Уоллер. — 6-е изд. — СПб.: Питер, 2023. — 960 с.
2. HTTP/1.1 RFC 9112 [Электронный ресурс]. — URL: <https://datatracker.ietf.org/doc/html/rfc9112> (дата обращения: 16.05.2026).
3. JWT RFC 7519 [Электронный ресурс]. — URL: <https://datatracker.ietf.org/doc/html/rfc7519> (дата обращения: 16.05.2026).
4. Fielding, R. T. Architectural Styles and the Design of Network-based Software Architectures: diss. / R. T. Fielding. — 2000.
5. Spring Boot Reference Documentation [Электронный ресурс]. — URL: <https://docs.spring.io/spring-boot/docs/current/reference/html/> (дата обращения: 16.05.2026).
6. PostgreSQL Documentation [Электронный ресурс]. — URL: <https://www.postgresql.org/docs/> (дата обращения: 16.05.2026).
7. nginx documentation [Электронный ресурс]. — URL: <https://nginx.org/en/docs/> (дата обращения: 16.05.2026).
8. Docker Documentation [Электронный ресурс]. — URL: <https://docs.docker.com/> (дата обращения: 16.05.2026).
9. Wireshark User's Guide [Электронный ресурс]. — URL: https://www.wireshark.org/docs/wsug_html_chunked/ (дата обращения: 20.05.2026).
10. ГОСТ 2.701-2008. ЕСКД. Схемы. Виды и типы. Общие требования. — Минск: Межгос. совет по стандартизации, 2009.
11. СТП 01-2024. Дипломные проекты (работы). Общие требования. — Минск: БГУИР, 2024. — 179 с.
12. ГОСТ 2.105-95. ЕСКД. Общие требования к текстовым документам. — Минск: Межгос. совет по стандартизации, 1996.

ПРИЛОЖЕНИЕ А. Листинг кода

Файл «docker-compose.yml»:

```
yaml
services:
  db:
    image: postgres:15-alpine
    environment:
      POSTGRES_DB: game2048
    ports:
      - "5432:5432"
    volumes:
      - pgdata:/var/lib/postgresql/data

  app:
    build:
      context: ./backend
    ports:
      - "8080:8080"
    environment:
      DB_HOST: db
      JWT_SECRET: change-this-secret-in-production-32bytes
      STORAGE_ROOT: /app/storage_data
    depends_on:
      - db

  nginx:
    image: nginx:1.27-alpine
    ports:
      - "80:80"
    volumes:
      - ./nginx/nginx.conf:/etc/nginx/conf.d/default.conf:ro
    depends_on:
      - app
# ... volumes: pgdata, storage_data
```

Файл «nginx/nginx.conf»:

```
nginx
upstream game_backend {
    server app:8080;
}

server {
    listen 80;
    location / {
        proxy_pass http://game_backend;
        proxy_set_header Host $host;
```

```

        proxy_set_header X-Real-IP $remote_addr;
        proxy_set_header X-Forwarded-For $proxy_add_x_forwarded_for;
    }
}

```

Файл «backend/.../controller/AuthController.java»:

```

java
@RestController
@RequestMapping("/api/auth")
public class AuthController {

    private final AuthService authService;

    @PostMapping("/register")
    public ResponseEntity<AuthResponse> register(@Valid @RequestBody RegisterRequest
request) {
        return ResponseEntity.ok(authService.register(request));
    }

    @PostMapping("/login")
    public ResponseEntity<AuthResponse> login(@Valid @RequestBody LoginRequest request) {
        return ResponseEntity.ok(authService.login(request));
    }
}

```

Файл «backend/.../service/AuthService.java»:

```

java
@Transactional
public AuthResponse register(RegisterRequest request) {
    if (userRepository.existsByUsername(request.getUsername())) {
        throw new IllegalArgumentException("Пользователь с таким именем уже существует");
    }
    User user = new User();
    user.setUsername(request.getUsername());
    user.setPasswordHash(passwordEncoder.encode(request.getPassword()));
    user = userRepository.save(user);
    String token = jwtUtil.generateToken(user.getUsername(), user.getId());
    return new AuthResponse(token, user.getUsername(), user.getId());
}

public AuthResponse login(LoginRequest request) {
    User user = userRepository.findByUsername(request.getUsername())
        .orElseThrow(() -> new IllegalArgumentException("Неверное имя пользователя или
пароль"));
    if (!passwordEncoder.matches(request.getPassword(), user.getPasswordHash())) {
        throw new IllegalArgumentException("Неверное имя пользователя или пароль");
    }
    String token = jwtUtil.generateToken(user.getUsername(), user.getId());
}

```

```

        return new AuthResponse(token, user.getUsername(), user.getId());
    }

```

Файл «backend/.../security/JwtUtil.java»:

```

java
public String generateToken(String username, Long userId) {
    return Jwts.builder()
        .subject(username)
        .claim("userId", userId)
        .issuedAt(new Date())
        .expiration(new Date(System.currentTimeMillis() + expirationMs))
        .signWith(key)
        .compact();
}

public boolean validateToken(String token) {
    try {
        getClaims(token);
        return true;
    } catch (JwtException | IllegalArgumentException e) {
        return false;
    }
}

```

Файл «backend/.../security/SecurityConfig.java»:

```

java
@Bean
public SecurityFilterChain filterChain(HttpSecurity http) throws Exception {
    http
        .csrf(csrf -> csrf.disable())
        .sessionManagement(session ->
            session.sessionCreationPolicy(SessionCreationPolicy.STATELESS))
        .authorizeHttpRequests(auth -> auth
            .requestMatchers("/api/auth/register", "/api/auth/login").permitAll()
            .requestMatchers("/api/records/top").permitAll()
            .requestMatchers("/api/profile", "/api/files/**").authenticated()
            .anyRequest().authenticated()
        )
        .addFilterBefore(jwtAuthFilter, UsernamePasswordAuthenticationFilter.class);
    return http.build();
}

@Bean
public PasswordEncoder passwordEncoder() {
    return new BCryptPasswordEncoder();
}

```

Файл «backend/.../security/JwtAuthFilter.java»:

```

java
@Override
protected void doFilterInternal(HttpServletRequest request,
                                HttpServletResponse response,
                                FilterChain filterChain) throws ServletException,
                                IOException {
    String token = getTokenFromRequest(request);
    if (StringUtils.hasText(token) && jwtUtil.validateToken(token)) {
        String username = jwtUtil.getUsernameFromToken(token);
        Long userId = jwtUtil.getUserIdFromToken(token);
        var auth = new UsernamePasswordAuthenticationToken(
            new UserPrincipal(userId, username), null, Collections.emptyList());
        SecurityContextHolder.getContext().setAuthentication(auth);
    }
    filterChain.doFilter(request, response);
}

private String getTokenFromRequest(HttpServletRequest request) {
    String bearer = request.getHeader("Authorization");
    if (StringUtils.hasText(bearer) && bearer.startsWith("Bearer ")) {
        return bearer.substring(7);
    }
    return null;
}

```

Файл «backend/.../controller/GameRecordController.java»:

```

java
@RestController
@RequestMapping("/api/records")
public class GameRecordController {

    @PostMapping
    public ResponseEntity<GameRecordDto> saveRecord(
        @AuthenticationPrincipal UserPrincipal principal,
        @Valid @RequestBody GameRecordRequest request) {
        return ResponseEntity.ok(gameRecordService.saveRecord(principal, request));
    }

    @GetMapping("/top")
    public ResponseEntity<List<GameRecordDto>> getTopScores(
        @RequestParam(defaultValue = "20") int limit) {
        return ResponseEntity.ok(gameRecordService.getTopScores(limit));
    }
}

```

Файл «backend/.../controller/FileStorageController.java»:

```

java

```

```

@PostMapping(value = "/avatars/me", consumes = MediaType.MULTIPART_FORM_DATA_VALUE)
public ResponseEntity<Map<String, Object>> uploadAvatar(
    @AuthenticationPrincipal UserPrincipal principal,
    @RequestParam("file") MultipartFile file) throws IOException {
    requirePrincipal(principal);
    storageService.storeAvatar(principal.getId(), ext, file.getInputStream(),
file.getSize());
    return ResponseEntity.ok(Map.of(
        "userId", principal.getId(),
        "avatarUrl", "/api/files/avatars/" + principal.getId()));
}

```

```

@PutMapping(value = "/saves/me", consumes = MediaType.APPLICATION_JSON_VALUE)
public ResponseEntity<Map<String, String>> uploadSave(
    @AuthenticationPrincipal UserPrincipal principal,
    @RequestBody byte[] body) throws IOException {
    requirePrincipal(principal);
    storageService.storeSave(principal.getId(), new ByteArrayInputStream(body),
body.length);
    return ResponseEntity.ok(Map.of("status", "saved", "savedAt",
Instant.now().toString()));
}

```

Файл «backend/.../filter/AccessControlFilter.java»:

```

java
@Override
protected void doFilterInternal(HttpServletRequest request,
                                HttpServletResponse response,
                                FilterChain filterChain) throws ServletException,
IOException {
    String clientIp = ApiLoggingFilter.clientIp(request);
    String path = request.getRequestURI();
    if (isIpBlocked(clientIp) || isPathBlocked(path)) {
        sendForbidden(request, response, "blocked");
        return;
    }
    if (isAuthPath(path) && isRateLimited(clientIp)) {
        sendForbidden(request, response, "Too many auth requests");
        return;
    }
    filterChain.doFilter(request, response);
}

private boolean isRateLimited(String ip) {
    int limit = Math.max(1, properties.getAuthRateLimitPerMinute());
    // ... подсчёт запросов в минуту по IP
    return bucket.count.incrementAndGet() > limit;
}

```

Файл «backend/.../filter/ApiLoggingFilter.java»:

```
java
private static final Logger log = LoggerFactory.getLogger("http.access");

@Override
protected void doFilterInternal(HttpServletRequest request,
                                HttpServletResponse response,
                                FilterChain filterChain) throws ServletException,
IOException {
    long start = System.currentTimeMillis();
    try {
        filterChain.doFilter(wrappedRequest, wrappedResponse);
    } finally {
        long duration = System.currentTimeMillis() - start;
        log.info("{} {} {} {}ms user={}",
                request.getMethod(),
                request.getRequestURI(),
                wrappedResponse.getStatus(),
                duration,
                resolveUser());
    }
}
```

Файл «backend/.../entity/User.java»:

```
java
@Entity
@Table(name = "users")
public class User {

    @Id
    @GeneratedValue(strategy = GenerationType.IDENTITY)
    private Long id;

    @Column(unique = true, nullable = false, length = 50)
    private String username;

    @Column(nullable = false, length = 255)
    private String passwordHash;

    @OneToMany(mappedBy = "user", cascade = CascadeType.ALL)
    private List<GameRecord> gameRecords = new ArrayList<>();
    // ... getters/setters
}
```

Файл: «backend/.../entity/GameRecord.java»

```
java
@Entity
```

```

@Table(name = "game_records")
public class GameRecord {

    @Id
    @GeneratedValue(strategy = GenerationType.IDENTITY)
    private Long id;

    @ManyToOne(fetch = FetchType.LAZY, optional = false)
    @JoinColumn(name = "user_id", nullable = false)
    private User user;

    @Column(nullable = false)
    private Integer score;

    @Column(nullable = false)
    private Integer maxTile;

    @Column(name = "played_at", nullable = false)
    private Instant playedAt;
    // ... movesCount, getters/setters
}

```

Файл «backend/.../storage/StoragePathResolver.java»:

```

java
public static Path resolve(Path root, String relativePath) {
    String normalized = decoded.replace('\\', '/');
    if (normalized.contains("..")) {
        throw new IllegalArgumentException("Недопустимый путь");
    }
    Path candidate = root.resolve(normalized).normalize();
    if (!candidate.toAbsolutePath().normalize().startsWith(rootNorm)) {
        throw new IllegalArgumentException("Путь выходит за пределы хранилища");
    }
    return candidate;
}

```

Файл: «frontend/game.js»:

```

javascript
function slideRow(row) {
    const line = row.filter(x => x !== 0);
    const merged = [];
    let i = 0;
    while (i < line.length) {
        if (i + 1 < line.length && line[i] === line[i + 1]) {
            merged.push(line[i] * 2);
            score += line[i] * 2;
            i += 2;
        } else {

```

```

        merged.push(line[i]);
        i += 1;
    }
}
while (merged.length < SIZE) merged.push(0);
// ... копирование в row
}

function move(direction) {
    if (over) return false;
    // ... сдвиг влево/вправо/вверх/вниз через slideRow и rotate
    if (moved) {
        movesCount++;
        addRandomTile();
        checkWin();
        checkGameOver();
    }
    return moved;
}

```

Файл «frontend/api.js»:

```

javascript
function headers(includeAuth = false, json = true) {
    const h = { 'Accept': 'application/json' };
    if (json) h['Content-Type'] = 'application/json';
    const token = getToken();
    if (includeAuth && token) {
        h['Authorization'] = 'Bearer ' + token;
    }
    return h;
}

async function request(method, path, body, auth = false) {
    const res = await fetch(BASE + path, {
        method,
        headers: headers(auth, body != null),
        body: body != null ? JSON.stringify(body) : undefined
    });
    // ... разбор ответа и ошибок
}

window.api = {
    async login(username, password) {
        return request('POST', '/auth/login', { username, password });
    },
    async saveRecord(score, maxTile, movesCount) {
        return request('POST', '/records', { score, maxTile, movesCount }, true);
    }
    // ... register, getTopScores, uploadAvatar, uploadGameSave
}

```



```
};
```

Файл «frontend/app.js»:

```
javascript
document.getElementById('auth-form').addEventListener('submit', async function (e) {
    e.preventDefault();
    const username = document.getElementById('auth-username').value.trim();
    const password = document.getElementById('auth-password').value;
    const fn = document.getElementById('modal-title').textContent === 'Регистрация'
        ? window.api.register : window.api.login;
    const res = await fn.call(window.api, username, password);
    window.api.setToken(res.token);
    closeModal();
    location.reload();
});

async function saveRecord() {
    if (!window.api.isAuthenticated()) {
        alert('Войдите, чтобы сохранить результат.');
```

```
        return;
    }
    await window.api.saveRecord(
        Game2048.getScore(),
        Game2048.getMaxTile(),
        Game2048.getMovesCount()
    );
    loadLeaderboard();
    loadMyRecords();
}
```

ПРИЛОЖЕНИЕ Б. Схема алгоритма работы системы